

Opracowanie

System e-commerce sklepu odzieżowego

Kacper Gumulak & Damian Judka

13.06.2026

Część 5: Testy systemu i zapewnienie jakości

Ostatni etap prac nad systemem e-commerce skupił się na rygorystycznej weryfikacji poprawności logicznej i wydajnościowej rdzenia bazy danych. Zastosowanie zautomatyzowanego frameworka testowego pozwoliło na wykrycie błędów, które były trudne do zidentyfikowania podczas manualnego przeglądu kodu SQL.

5.1 Metodologia i automatyzacja testów

Do przeprowadzenia audytu wykorzystano bibliotekę pytest w środowisku Python 3.12. Testy zostały zaprojektowane jako proces całkowicie odizolowany, komunikujący się z bazą danych MariaDB 11 za pośrednictwem sterownika PyMySQL. Każdy test operuje w ramach dedykowanej sesji, co pozwala na wielokrotne uruchamianie scenariuszy bez wpływu na dane produkcyjne.

Kategoria testu	Liczba przypadków	Cel testowania
Jednostkowe (Unit)	4	Weryfikacja unikalności danych, poprawnego generowania haseł i walidacji ilościowej.
Modułowe (Integration)	2	Badanie współpracy procesów: odrzucanie pustych koszyków, blokada oversellingu.
Funkcjonalne (E2E)	2	Pełna ścieżka: zakup -> płatność -> faktura -> zwrot (Event Sourcing).

5.2 Analiza wykrytych błędów i wdrożone poprawki

Proces testowania obnażył trzy krytyczne obszary wymagające interwencji programistycznej i architektonicznej.

A. Błąd zarządzania transakcjami (Error 1644)

Podczas testów integracyjnych wykryto konflikt między sterownikiem aplikacji a logiką procedur składowanych.

```
E pymysql.err.OperationalError: (1644, "Transaction characteristics can't be changed while a transaction is in progress")
```

Przyczyna: Próba zmiany poziomu izolacji (READ COMMITTED) wewnątrz procedury, podczas gdy Python utrzymywał już aktywną transakcję.

Naprawa: Przetęczenie sterownika w tryb autocommit, delegując pełną kontrolę nad transakcyjnością silnikowi bazy danych.

B. Luka w integralności referencyjnej (IntegrityError 1451)

Testy jednostkowe wykazały błąd podczas próby usuwania kont użytkowników.

```
E pymysql.err.IntegrityError: (1451, "Cannot delete or update a parent row: a foreign key constraint fails (`cart`, CONSTRAINT `customer_fk2`)")
```

Naprawa: Wykryto brak kaskadowego usuwania. Wdrożono poprawkę architektoniczną w schemacie SQL:

```
ALTER TABLE `cart` ADD CONSTRAINT `customer_fk2` FOREIGN KEY  
    (`customer_id`) REFERENCES `customers` (`customer_id`) ON DELETE CASCADE;
```

C. Konflikt kolacji znaków (Collation Mismatch)

W środowisku Docker (MariaDB 11) wystąpił problem z porównywaniem ciągów tekstowych.

```
E OperationalError: (1644, "Illegal mix of collations (utf8mb4_general_ci) and (utf8mb4_uca1400_ai_ci) for operation '='")
```

Naprawa: Ujednolicono standard kodowania serwera poprzez parametry startowe w Docker

```
Compose: --character-set-server=utf8mb4 --collation-server=utf8mb4_general_ci.
```

5.3 Podsumowanie wyników

Status końcowy: 8/8 PASSED

Wszystkie zdefiniowane scenariusze testowe, w tym zaawansowana weryfikacja mechanizmu Event Sourcing po anulowaniu zamówienia, zakończyły się sukcesem.